1

Databases 1 · Week 1

Database management systems

File-based systems duplicated data, so copies became inconsistent.

- Programs depended on file layouts: a layout change meant rewriting code.
- A DBMS stores data once, describes it, protects it, and shares it.

> **Think of it like this.** Picture every office in a company keeping its own paper address book. When a customer moves, someone has to update every book, and some will be missed. A database is one shared, well-organised address book that everyone consults.

Data + context = information; people turn information into knowledge.

- A database models a mini-world, is coherent, and has a purpose.
- The catalogue stores metadata: the structure and rules of the data.

> **Think of it like this.** A single puzzle piece is data. Seeing that it is part of the sky is information. Knowing where to start the puzzle next time is knowledge. The box with the picture on the lid is the metadata.

A DBMS defines, constructs, manipulates, and shares databases.

- Database system = DBMS + data + application programs.
- Benefits: less redundancy, controlled sharing, security, efficient queries, recovery, and integrity.
- Tradeoffs: setup, hardware, training, and administration costs; simple files can suit small, stable, single-user data.

> **Think of it like this.** A DBMS is like a librarian. You do not walk into the storage room yourself: you ask, and the librarian finds the book quickly, checks you may borrow it, records the loan, and makes sure two people do not take the last copy.

DBA: access, performance, security. Designer: structure, rules, views.

- Analysts specify; application programmers build and maintain.
- End users can be casual, parametric, sophisticated, or stand-alone.

> **Think of it like this.** Think of a theatre: the designer draws the stage, the administrator runs the building and holds the keys, programmers build the props, and the audience, the end users, are why the show exists.

Relation = table, attribute = column, tuple = row.

- Schema = structure; data (instance) = content at a moment.
- Other models exist: hierarchical, network, object-oriented, NoSQL.

> **Think of it like this.** A table is like a well-designed form printed many times: the questions on the form are the columns (the schema), and each filled-in copy is a row (the data).

PK: unique and not null; it identifies each row.

- FK: stores another table's PK; it creates the relationship.
- N:M relationships need a junction table with two FKs.

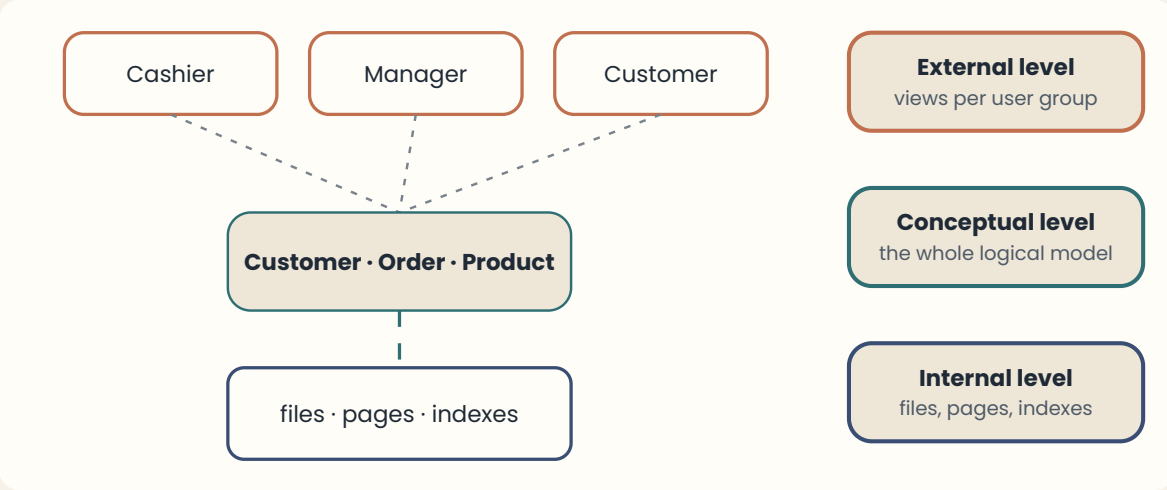
> **Think of it like this.** A primary key is like a national ID number: one per person, never shared. A foreign key is that same number written on a form somewhere else, so you can always find the person it refers to.

Constraints: PRIMARY KEY, FOREIGN KEY, NOT NULL, CHECK.

- Transaction = all or nothing: COMMIT or ROLLBACK.
- Concurrency control, logs, backups, and least privilege keep data safe.

> **Think of it like this.** A transaction is like a bank transfer: money leaves one account and arrives in the other, or nothing happens. You never want the money to leave and not arrive.

External = user views; conceptual = logical model; internal = physical storage.



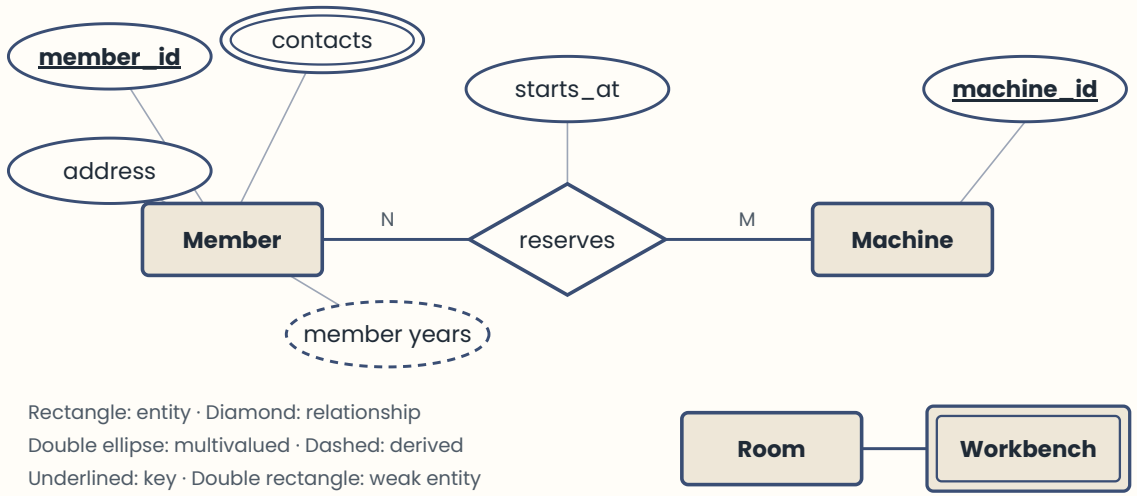
> **Think of it like this.** A restaurant: diners see the menu (external), the chef works from recipes (conceptual), and the pantry is organised in shelves and fridges (internal). Reorganising the pantry should not change the menu.

SELECT columns FROM table WHERE condition.

- ORDER BY sorts; JOIN … ON combines tables through keys.
- UPDATE and DELETE always need a WHERE clause.

> **Think of it like this.** SELECT is like asking a librarian: "show me the titles and authors (SELECT) from the history shelf (FROM) published after 2000 (WHERE)".

Conceptual (ER) → logical (tables) → physical (SQL).



> **Think of it like this.** The three stages are like building a house: the sketch you discuss with the family (conceptual), the architect's floor plan with measurements (logical), and the construction itself with real materials (physical).

Entity → table; composite → several columns; multivalued → new table; derived → not stored.

- 1:N → FK on the N side; N:M → junction table; 1:1 → FK on either side.
- Keys can be natural, surrogate, or composite.

> **Think of it like this.** The mapping rules are a recipe: the ER diagram is the list of ingredients, and the rules tell you exactly which pot each one goes into.

DDL: CREATE, ALTER, DROP, TRUNCATE. DML: INSERT, SELECT, UPDATE, DELETE.

- Use DECIMAL for money, VARCHAR for text, DATE for dates.
- ON DELETE: CASCADE, RESTRICT, or SET NULL, chosen per relationship.

> **Think of it like this.** DDL builds the shelves; DML puts things on them, moves them, and takes them off. TRUNCATE empties the shelves; DROP takes them out of the room.

Both are relational servers and share core SQL concepts.

- Their dialects differ; check generated-id, date, JSON, and DDL syntax.
- The browser SQL Lab uses SQLite, not either server.

> **Think of it like this.** SQL is like a shared language with regional accents: you can understand the basic conversation, but check the local phrasebook for commands that vary.

Linux: install with the package manager and check the system service.

- Windows: use the official installer, set a local admin password, and keep the default port if available.
- Create a database, then connect to localhost with the matching client.

> **Think of it like this.** Installing the DBMS is setting up a workshop; connecting with pgAdmin, Workbench, psql, or mysql is choosing a door into it. The SQL Lab is a separate portable workbench inside the browser.

Step by step

Put the sale in order

1. BEGIN the transaction
2. Insert the new order
3. Insert the order item
4. Reduce the product stock
5. COMMIT if every step worked

Check yourself

1. What is the main risk of storing the same customer in several files?

A) The copies become inconsistent B) The files load faster C) Programs need less code

2. Which statement is information, not just data?

A) "Customer 301 ordered a sourdough loaf" B) "Sourdough loaf" C) "301"

3. A data analyst writes her own SQL queries and small tools to study sales. What kind of end user is she?

A) Sophisticated B) Parametric C) Casual

Answers are in the speaker notes.

Practise next

Labs

- First queries in the SQL Lab
- Join customers and orders
- See integrity and transactions at work
- Design the bakery in the Design Studio

On the site

- Design Studio
- SQL Lab